

Student Performance Analysis System — Project Documentation

Project: Factors Influencing Student Performance with SQL (Project 6)

Deliverable: A data-driven web application that ingests any tabular dataset, cleans it, loads it into a SQL database, and produces analytical insights (with the SQL that proves them), interactive charts, and a downloadable PDF report.

Live deployment: <https://student-performance-analysis-8uff.onrender.com>

Source code: GitHub — [amier155555-max/Student-Performance-Analysis-SQL-System](https://github.com/amier155555-max/Student-Performance-Analysis-SQL-System)

This document maps the standard university deliverables onto what the project actually implements today.

1. Project Planning & Management

1.1 Project Proposal — Overview, Objectives & Scope

Overview. A web-based SQL Analytics Studio that turns a raw data file into cleaned, queryable data and evidence-backed insights without the user writing any SQL. It was designed around the UCI Student-Performance dataset but generalised to work with any dataset that has at least one numeric outcome column.

Objectives.

- Load educational (and generic) datasets into a relational database.
- Clean and pre-process the data reproducibly.
- Explore key relationships with fixed, auditable SQL queries.
- Test hypotheses about the factors influencing performance.
- Package the analysis as repeatable stored procedures and a nightly batch.
- Document insights automatically, backed by real SQL and charts.

Scope (in). File upload (CSV/TSV/XLSX), automated cleaning, SQLite storage, normalized education schema, hypothesis dashboard, general auto-analysis for arbitrary datasets, stored-procedure/batch runs, activity logging, PDF documentation, public deployment.

Scope (out). User accounts/authentication, multi-tenant data isolation, real-time streaming ingestion, and a managed cloud database (SQLite is used; a PostgreSQL design is provided for future scale).

1.2 Project Plan — Timeline, Milestones, Deliverables

Phase	Deliverable	Status
Planning & Management	Proposal, plan, roles, risk plan, KPIs	Done
Literature Review	Related work, evaluation, grading criteria	Done

Phase	Deliverable	Status
Requirements Gathering	Stakeholders, use cases, functional/non-functional	Done
System Analysis & Design	ERD, schema, DFD, class/sequence/activity/state, UI/UX	Done
Implementation	Flask app (07-Implementation_Source_Code/spa)	Done
Deployment & Stored Procedures	08-Deployment_Stored_Procedures/	Done
Automation & Monitoring	09-Automation_Monitoring/ + in-app logs	Done
Final Docs, Demo & Presentation	10-Final_Presentation_and_Reports/ + live URL	Done

1.3 Task Assignment & Roles

Role	Responsibility
Project Lead	Scope, milestones, integration of all parts
Data Engineer	Cleaning pipeline (data_cleaner.py), schema mapping
Database Engineer	Schema design (schema.sql), stored procedures
Backend Developer	Flask routes, analytics, report/PDF generation
Frontend/UX	Templates, styling, Chart.js visualisations
DevOps	GitHub, Render deployment, automation scripts

1.4 Risk Assessment & Mitigation

Risk	Impact	Mitigation (as implemented)
SQL injection from user data	High	All analytics SQL is fixed/developer-authored; table/column names are standardised; no free-text SQL from users.
Malformed / unexpected uploads	Medium	Robust reader (multiple delimiters/encodings), friendly UnsupportedFileError, per-request try/except.
Uploaded file doesn't match schema	Medium	Generic fallback: flat cleaned_data table + auto relationship analysis.
Free-tier data loss on restart	Medium	Documented; SPA_DATA_DIR made configurable for a persistent disk; PostgreSQL design ready for future.
Broken charts (CDN outage)	Low	Chart.js vendored locally; JS guard shows a message instead of a blank canvas.

	Risk	Impact	Mitigation (as implemented)
	Large files exhausting memory	Low	50 MB upload cap; profiling uses a sampled slice.

1.5 KPIs (Key Performance Indicators)

- Ingestion success rate — % of uploaded files cleaned & loaded without error.
- Schema-match rate — % of uploads recognised as the student schema vs generic.
- Analysis success — stored-procedure runs succeeded / total (target: 8/8 for student data).
- Response time — dashboard renders in < 2 s on the sample datasets.
- Uptime / availability — /health endpoint returns ok (monitored by the platform).
- Report completeness — every insight in the PDF is backed by its SQL and real numbers.

2. Literature Review

2.1 Related Work & Positioning

The project builds on the well-known UCI "Student Performance" dataset (Cortez & Silva), which links demographic, social, and study-related attributes to final grades (G1–G3). Prior work typically explores this data in notebooks (pandas/scikit-learn). This project differs by delivering a self-service SQL analytics application: the analysis is expressed as fixed SQL over a normalized schema, is auditable, repeatable (stored procedures), automatable (nightly batch), and is generalised beyond the original dataset to any tabular file with a numeric outcome.

2.2 Feedback & Evaluation (criteria the project targets)

- Correctness — cleaning and queries produce accurate, reproducible numbers.
- Data-driven — every reported insight is generated live from the current DB.
- Generality — works on arbitrary datasets, not only students.
- Security — injection-safe query layer.
- Deployment — publicly accessible, runs on any device via the browser.

2.3 Suggested Improvements (future work)

- Persistent storage (Render disk or managed PostgreSQL) so data survives restarts.
- User authentication and per-user datasets.
- Correlation/regression significance testing on the generic analysis.
- Export of the report to .docx/.pptx in addition to PDF.
- CI/CD (automated tests on push before deploy).

2.4 Final Grading Criteria (self-assessment breakdown)

Area	Weight	Evidence
Documentation	25%	Folders 01–10 + this document

Area	Weight	Evidence
Implementation	35%	Working Flask app, general analysis, deployment
Testing & QA	20%	tests/test_cleaner.py, run logs, manual validation
Presentation / Demo	20%	Live URL + slide deck in folder 10

3. Requirements Gathering

3.1 Stakeholder Analysis

Stakeholder	Needs
Analyst / End user	Upload a file and get insights without writing SQL
Educator / Decision-maker	Understand which factors affect performance
Data engineer	Trustworthy, reproducible cleaning + audit trail
Instructor / Evaluator	Clear documentation and a working live demo

3.2 User Stories & Use Cases

- As an analyst, I upload a CSV so that it is cleaned and stored automatically.
- As an analyst, I view a preview of exactly what the cleaning changed so I can trust it.
- As a decision-maker, I open the dashboard to see which factors move the outcome.
- As an analyst, I download a PDF report with insights and the SQL behind them.
- As an operator, I re-run the analysis on demand (stored procedures / batch) and see an audit log.
- As an operator, I review the activity log to see what happened and when.

Primary use cases: Upload & Clean, Preview Cleaning Report, View Dashboard, Download Documentation, Run Stored-Procedure Analysis, Run Nightly Batch, View Activity Log, Configure Settings, Reset Database.

3.3 Functional Requirements

1. Accept CSV/TSV/XLSX uploads up to 50 MB from any device via the browser.
2. Clean data: standardise column names, trim whitespace, drop empty rows/columns, remove duplicates, coerce numeric types, fill missing values, cap outliers (IQR).
3. Always store cleaned data in a flat cleaned_data table.
4. If columns match the student schema, populate the normalized 5-table schema.
5. Run fixed hypothesis queries and render them as bar/line/pie/doughnut charts.
6. For any non-student file, auto-detect a numeric outcome and analyse every other column against it (categorical average; numeric quartile trend).
7. Generate a downloadable PDF report (schema, steps, insights + SQL + charts).
8. Provide stored-procedure-style repeatable analysis and a nightly batch job, both writing to an audit/results table.

9. Record an activity log of uploads, runs, downloads, and resets.
10. Expose a /health endpoint for monitoring.

3.4 Non-Functional Requirements

- Security: injection-safe (fixed SQL, SELECT-only query path, secret via env var).
- Usability: no SQL knowledge required; clear flash messages; responsive UI.
- Reliability: every request is guarded; logging never breaks a request.
- Performance: dashboard < 2 s on samples; profiling uses a sampled slice.
- Portability: runs locally (python app.py) or on any WSGI host (gunicorn).
- Maintainability: modular code, single-responsibility modules, documented.

4. System Analysis & Design

4.1 Problem Statement & Objectives

Educational data holds signals about what drives student performance, but raw files are messy and analysing them requires SQL/programming skill. The system solves this by automating cleaning, storage, and evidence-backed analysis behind a simple web UI — and generalises the approach to any tabular dataset.

4.2 Use Case Diagram & Descriptions

Actor: User (analyst/operator). System: SQL Analytics Studio.

Key use cases and their flows:

- Upload & Clean: User selects a file system validates, cleans, loads, and redirects to the cleaning preview.
- View Dashboard: System runs the applicable analysis (student hypotheses or generic auto-analysis) and renders charts + KPIs.
- Download Documentation: System builds a live Markdown report and renders it to a charted PDF.
- Run Stored Procedures / Batch: System executes the analyses, persists results, and logs each run.

(Diagrams: 04-System_Analysis_and_Design/Usecase.png, usecase2.png.)

4.3 Functional & Non-Functional Requirements

Summarised in §3.3–3.4. Capabilities: ingest clean store analyse report automate. Constraints: injection-safe, browser-only access, 50 MB cap.

4.4 Software Architecture

Style: a layered MVC-style Flask application.

- View (templates/ + static/): Jinja2 pages (upload, preview, dashboard, deployment, logs, settings) and Chart.js visualisations.
- Controller (app.py): HTTP routes that orchestrate the flow and handle errors.
- Model / Logic (modules/):

- data_cleaner.py — reusable cleaning pipeline.
- schema_mapper.py — maps cleaned columns onto the normalized schema.
- db_manager.py — safe SQLite read/write.
- analytics.py — fixed hypothesis queries + generic auto-analysis.
- report_builder.py — builds the Markdown report.
- pdf_report.py — renders the report to a charted PDF.
- deployment.py — stored-procedure/batch bridge.
- activity_log.py — activity/audit log.
- Config (config.py): settings + data paths (SPA_DATA_DIR overridable).
- Deployment: Render web service running gunicorn app:app (see render.yaml).

Technology stack. Python 3.12, Flask, gunicorn, pandas, numpy, openpyxl, SQLite, reportlab (+ arabic-reshaper/python-bidi) for PDF, Chart.js (vendored) for charts. Hosted on Render; source on GitHub.

4.5 Database Design & Data Modeling

Normalized schema (production / PostgreSQL — 04-.../schema.sql):

five core entities linked by student_id — student, family, enrollment, study_behavior, grade — plus lookup tables (school, job_type, reason_type, guardian_type). Grades are modelled 1:N (period 1/2/3, score 0–20) with keys, checks, and indexes.

Runtime store (SQLite): the same five tables (values stored inline to simplify auto-import) for matched student data; a flat cleaned_data table for any other dataset. Stored-procedure runs persist to analysis_run_log and analysis_results.

- Logical schema: entities, attributes, PK/FK relationships (see ERD).
- Physical schema: SQLite tables created at runtime via to_sql; PostgreSQL DDL in schema.sql.
- Normalization: 3NF in the relational design; repeated text factored into lookup tables in the PostgreSQL version.

(Diagram: 04-System_Analysis_and_Design/ERD.png.)

4.6 Data Flow & System Behavior

DFD (context detail): User Upload Clean (data_cleaner)

Store (db_manager cleaned_data [+ normalized tables]) Analyse (analytics) Present (dashboard charts / PDF report). The nightly batch reads the store and writes a JSON report + run log.

(Diagrams: DFD_level0.png, DFD_level1.png.)

- Sequence diagram — Upload flow: browser /upload DataCleaner.run schema_mapper db_manager.write_* redirect /preview.

(Sequence.png, Sequence2.jpeg.)

- Activity diagram — the ETL pipeline: receive file validate clean load flat (match? normalize) analyse show/report. (Activity.png.)

- State diagram — a dataset's states: *Uploaded Cleaned Stored

Analysed Reported (and Reset back to empty). (State.png.)*

- Class diagram — modules as collaborating classes/functions: DataCleaner & CleaningReport, schema_mapper, db_manager, analytics, report_builder/pdf_report, deployment, activity_log. (Class1.png, Class2.png.)

4.7 UI/UX Design & Prototyping

- Wireframes/mockups: upload (drag-and-drop), preview (KPI cards + report), dashboard (chart grid), deployment, logs, settings. (05-UI_UX_and_API/UI.png, UI 2.png, UI 3.jpeg.)
- UI/UX guidelines: a calm "sage / warm paper" palette, Fraunces + IBM Plex typography, consistent cards, accessible contrast, and clear status feedback via flash messages. Charts use a shared colour palette across the app and PDF.

4.8 System Deployment & Integration

- Technology stack: Flask (backend + server-rendered frontend), SQLite (database), gunicorn (WSGI server), Render (hosting), GitHub (VCS).
- Deployment diagram: GitHub repo Render build (pip install) Render web service (gunicorn app:app on \$PORT) public HTTPS URL any browser.
- Component diagram: Browser (HTML/Chart.js) Flask app (routes) modules (cleaning/analytics/report) SQLite DB; deployment bridge stored-procedure layer; nightly job reports directory.

4.9 Additional Deliverables

- API / endpoints: GET / (upload), POST /upload, GET /preview, GET /dashboard, GET /deployment + POST /deployment/run-procedures & /run-batch, GET /logs + POST /logs/clear, GET/POST /settings, POST /database/reset, GET /download/csv|sql|documentation, GET /health.
- Testing & validation: unit test for the cleaner (tests/test_cleaner.py); end-to-end validation via the run log (8/8 student procedures succeed) and live-server upload checks; generic analysis verified on student, exam-score, and sales datasets.
- Deployment strategy: hosted on Render (free tier — spins down on inactivity, ephemeral filesystem). Scaling path: attach a persistent disk (set SPA_DATA_DIR) or migrate to managed PostgreSQL (schema already provided). Auto-deploy from GitHub can be enabled for CI-style delivery.